

C# - Phần 10.10: LINQ Join() cơ bản

Lý thuyết - Cú pháp - Kiểu dữ liệu - Bài tập - Đáp án - Đánh giá

Mục tiêu: Hiểu Join() dùng để nối hai collection theo khóa chung, phân biệt outer/inner key, hiểu resultSelector, trường hợp không match/match nhiều phần tử, kiểu dữ liệu kết quả và mối liên hệ với INNER JOIN trong SQL.

PHẦN A - BẢN CHẤT CỦA JOIN()

1. Join() dùng để làm gì?

Join() dùng để kết nối các phần tử từ hai collection khi khóa của chúng có giá trị khớp nhau.

Students	Registrations
Id	StudentId
S01 -----	S01
S02 -----	S02

Điều kiện nối trong ví dụ là Student.Id == Registration.StudentId.

2. Ví dụ dữ liệu

```
List<Student> students = new List<Student>
{
    new Student("S01", "An"),
    new Student("S02", "Binh"),
    new Student("S03", "Cuong"),
    new Student("S04", "Dung")
};

List<Registration> registrations =
    new List<Registration>
    {
        new Registration("S01", "C#"),
        new Registration("S01", "SQL"),
        new Registration("S02", "Java"),
        new Registration("S04", "ASP.NET"),
        new Registration("S05", "Python")
    };
};
```

3. Cú pháp cơ bản

```
var result =
    students.Join(
        registrations,
        student => student.Id,
        registration => registration.StudentId,
        (student, registration) =>
            $"{student.Name} - {registration.ObjectName}"
    );
```

Lưu ý: Trong bài thực hành, property CourseName được đổi thành ObjectName. Điều này không thay đổi bản chất của Join().

4. Năm thành phần cần đọc

```
outer.Join(
    inner,
    outerKeySelector,
    innerKeySelector,
    resultSelector
);
```

- outer: collection thứ nhất - ở đây là students.
- inner: collection thứ hai - ở đây là registrations.

- outerKeySelector: lấy khóa bên Student, student => student.Id.
- innerKeySelector: lấy khóa bên Registration, registration => registration.StudentId.
- resultSelector: quyết định mỗi cặp match sẽ trở thành dữ liệu gì trong kết quả.

PHẦN B - KHÓA VÀ CƠ CHẾ MATCH

5. Hai khóa phải tương thích

```
student => student.Id
→ Student -> string

registration => registration.StudentId
→ Registration -> string
```

Join() phải so sánh hai khóa để xác định match, nên chúng cần có kiểu tương thích.

6. Một Student có thể match nhiều Registration

```
Student:
S01 - An

Registrations:
S01 - C#
S01 - SQL

Kết quả:
An - C#
An - SQL
```

Mỗi lần match tạo ra một phần tử kết quả. Join() không tự Distinct() kết quả.

7. Phần tử không match

Join() cơ bản trong LINQ có tư duy giống INNER JOIN: chỉ các cặp có khóa match mới xuất hiện.

```
S03 có Student nhưng không có Registration
→ không xuất hiện

S05 có Registration nhưng không có Student
→ không xuất hiện
```

PHẦN C - RESULT SELECTOR

8. resultSelector dùng để làm gì?

resultSelector quyết định dữ liệu được tạo ra từ mỗi cặp Student + Registration vừa match.

```
(student, registration) =>
    $"{student.Name} - {registration.ObjectName}"
```

Kết quả mỗi cặp là string, nên toàn bộ Join trả về IEnumerable<string>.

```
(student, registration) =>
    student.Name
```

Nếu chỉ trả student.Name thì kết quả vẫn là IEnumerable<string>. Nếu một Student match nhiều Registration, tên có thể xuất hiện nhiều lần.

```
(student, registration) => new
{
    StudentName = student.Name,
    ObjectName = registration.ObjectName
}
```

Trường hợp này tạo anonymous object. Kết quả là IEnumerable của một anonymous type.

9. Ví dụ kết quả anonymous object

```
foreach (var item in result3)
{
```

```

    Console.WriteLine(
        $"{item.StudentName} - {item.ObjectName}");
}

```

Dùng var rất phù hợp với anonymous type vì không có tên class tường minh để viết trực tiếp.

PHẦN D - KIỂU DỮ LIỆU VÀ PIPELINE

10. result1

```

var result1 =
    students.Join(
        registrations,
        student => student.Id,
        registration => registration.StudentId,
        (student, registration) =>
            $"{student.Name} - {registration.ObjectName}"
    );

→ IEnumerable<string>

```

11. result2

```

var result2 =
    students.Join(
        registrations,
        student => student.Id,
        registration => registration.StudentId,
        (student, registration) => student.Name
    );

→ IEnumerable<string>

```

12. result3

```

var result3 =
    students.Join(
        registrations,
        student => student.Id,
        registration => registration.StudentId,
        (student, registration) => new
        {
            StudentName = student.Name,
            ObjectName = registration.ObjectName
        }
    );

→ IEnumerable<anonymous type>

```

13. Join() không sửa collection gốc

students và registrations vẫn giữ nguyên sau Join(). Join() chỉ tạo một sequence kết quả mới.

14. Liên hệ với SQL

```

LINQ:
students.Join(
    registrations,
    student => student.Id,
    registration => registration.StudentId,
    ...)

```

```

SQL tư duy tương đương:
FROM Students s
INNER JOIN Registrations r
    ON s.Id = r.StudentId

```

Bài này chỉ cần hiểu tư duy khóa bên A = khóa bên B; chưa cần học sâu SQL JOIN.

PHẦN E - CÁCH ĐỌC JOIN() KHÔNG BỊ RỐI

1. Collection thứ nhất là gì?
→ students
2. Nối với collection nào?
→ registrations
3. Khóa bên Student?
→ Id
4. Khóa bên Registration?
→ StudentId
5. Match xong tạo dữ liệu gì?
→ do resultSelector quyết định

Câu nhớ nhanh: Join() = nối hai collection bằng khóa chung, rồi quyết định dữ liệu kết quả muốn lấy từ hai bên.

PHẦN F - BÀI TẬP 10.10

- Dùng Join() nối Student.Id với Registration.StudentId, tạo string 'StudentName - ObjectName'.
- Dùng foreach hiển thị toàn bộ kết quả.
- Giải thích vì sao S03 không xuất hiện.
- Giải thích vì sao Registration S05 - Python không xuất hiện.
- Giải thích vì sao S01 xuất hiện hai lần.
- Dùng Join() nhưng resultSelector chỉ trả student.Name.
- Xác định kiểu kết quả của câu trên và giải thích vì sao An xuất hiện hai lần.
- Dùng Join() tạo anonymous object gồm StudentName và ObjectName.
- Kiểm tra students và registrations gốc không bị thay đổi.
- Tự giải thích kiểu dữ liệu của từng biến var.

PHẦN G - ĐÁP ÁN CODE

```
public class Program
{
    public static void Main(string[] args)
    {
        List<Student> students = new List<Student>
        {
            new Student("S01", "An"),
            new Student("S02", "Binh"),
            new Student("S03", "Cuong"),
            new Student("S04", "Dung")
        };

        List<Registration> registrations =
            new List<Registration>
            {
                new Registration("S01", "C#"),
                new Registration("S01", "SQL"),
                new Registration("S02", "Java"),
                new Registration("S04", "ASP.NET"),
                new Registration("S05", "Python")
            };

        var result1 =
            students.Join(
                registrations,
                student => student.Id,
                registration => registration.StudentId,
                (student, registration) =>
                    $"{student.Name} - {registration.ObjectName}"
            );
    }
}
```

```

foreach (string item in result1)
{
    Console.WriteLine(item);
}

var result2 =
    students.Join(
        registrations,
        student => student.Id,
        registration => registration.StudentId,
        (student, registration) => student.Name
    );

foreach (string item in result2)
{
    Console.WriteLine(item);
}

var result3 =
    students.Join(
        registrations,
        student => student.Id,
        registration => registration.StudentId,
        (student, registration) => new
        {
            StudentName = student.Name,
            ObjectName = registration.ObjectName
        }
    );

foreach (var item in result3)
{
    Console.WriteLine(
        $"{item.StudentName} - {item.ObjectName}");
}

foreach (Student student in students)
{
    Console.WriteLine(student);
}

foreach (Registration registration in registrations)
{
    Console.WriteLine(registration);
}
}

```

PHẦN H - CÂU HỎI LÝ THUYẾT

1. Join() dùng để làm gì?
2. Join() cơ bản làm việc với bao nhiêu collection?
3. Trong ví dụ Student/Registration, khóa bên Student là gì?
4. Khóa bên Registration là gì?
5. Vì sao kiểu của hai khóa phải tương thích?
6. resultSelector dùng để làm gì?
7. Nếu một Student không có Registration phù hợp thì chuyện gì xảy ra?
8. Nếu một Student match với ba Registration thì có bao nhiêu phần tử kết quả liên quan?
9. Join() có sửa collection gốc không?
10. Hãy giải thích từng thành phần trong lời gọi students.Join(...).

PHẦN I - ĐÁP ÁN LÝ THUYẾT

1. Join()

Kết nối các phần tử từ hai collection dựa trên hai khóa có giá trị khớp nhau.

2. Số collection

Cơ bản là hai collection: outer và inner.

3. Khóa bên Student

Student.Id.

4. Khóa bên Registration

Registration.StudentId.

5. Kiểu khóa

Hai khóa cần tương thích vì Join() phải so sánh chúng để xác định match.

6. resultSelector

Quyết định mỗi cặp outer + inner đã match sẽ trở thành dữ liệu gì trong collection kết quả.

7. Không có match

Student đó không tạo ra phần tử nào trong kết quả Join() cơ bản.

8. Match ba Registration

Có ba phần tử kết quả liên quan tới Student đó.

9. Collection gốc

Không bị thay đổi.

10. Thành phần Join

students = outer; registrations = inner; student.Id = outer key; registration.StudentId = inner key; lambda cuối = resultSelector.

PHẦN J - ĐÁNH GIÁ BÀI LÀM

- Phần code: khoảng 9.4/10.
- Phần lý thuyết: khoảng 9.1/10.
- Tổng thể: khoảng 9.2-9.3/10 - ĐẠT.
- Đã hiểu outer collection và inner collection.
- Đã chọn đúng Student.Id và Registration.StudentId làm khóa.
- Đã hiểu phần tử không match sẽ không xuất hiện trong Join() cơ bản.
- Đã hiểu một Student có thể tạo nhiều kết quả nếu match nhiều Registration.
- Đã tạo đúng kết quả string và anonymous object.
- Điểm cần củng cố: resultSelector quyết định mỗi cặp match trở thành dữ liệu gì.
- Điểm còn thiếu trong code ban đầu: thực sự in lại students và registrations để chứng minh collection gốc không đổi.
- Typo nhỏ: IEnumerable<sting> phải là IEnumerable<string>.

1. Bốn ý cốt lõi phải nhớ

```
Join
→ 2 collection

outer key
→ khóa bên collection thứ nhất

inner key
→ khóa bên collection thứ hai

resultSelector
→ quyết định dữ liệu kết quả
```

2. Kết luận

Kết quả: Phần 10.10 - Join() cơ bản đã ĐẠT. Bài tiếp theo là 10.11 - Bài tổng hợp LINQ.